



LE WEB AVEC UN TRUC EN PLUS EN 2.0



Formation JavaScript, les fondamentaux



PRÉSENTATION LOGISTIQUE

ORSYS & VOUS



Bienvenue chez Orsys

Présentation



- Indépendant
 - Prestataire d'Orsys
- Animateur
 - Monsieur DESORBAIX Alexandre
- Développeur
 - software, client lourd ,serveur
 - » c, c++, c#, java, ...
 - web, client léger
 - » html, php, sql, js, css,...
- blogger sous Wordpress



Bienvenue chez Orsys
logistique



- Nombre de jours :
 - 3 jours
- Horaires
 - Début : 9h
 - Fin : 17h30 approx.
- Pause
 - 10h30 approx. Matin
 - 12h30 approx. Midi
 - 15h30 approx. Après-midi



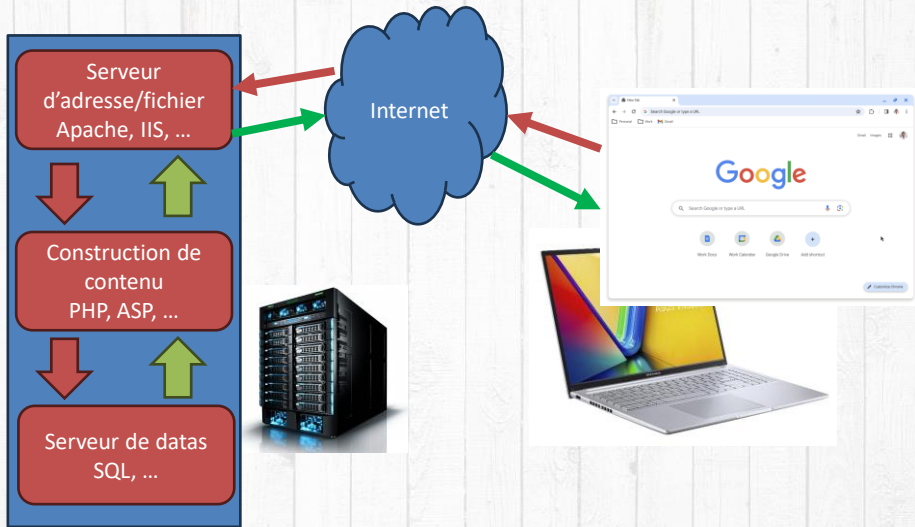
LES TECHNOLOGIES DU WEB



Architecture du WEB

- Le web se décompose en 2 ensembles:
 - Serveur
 - Php, asp, node, python, perl, ruby, ...
 - Serveur de dynamisation de contenu
 - Api
 - Client
 - HTML, CSS, JS
 - Navigateur
 - Chrome / chromium (EDGE) / chrome CANARY
 - Firefox
 - Opera, Brave,

Architecture WEB



Architecture du WEB

- Le web se décompose en 2 ensembles:
 - Serveur
 - Répond à une adresse URL
 - » Apache, IIS, Node, ...
 - Assemble des données depuis une base de données
 - » Sql, ...
 - Met en œuvre du code en fonction des demandes des clients de navigation.
 - Pages complètes
 - Segments de pages
 - Datas (json/xml)
 - » Langage de dynamisation (PHP, ASP, NODE,...)
 - Diffuse le contenu
 - » Apache, Node, IIS,...



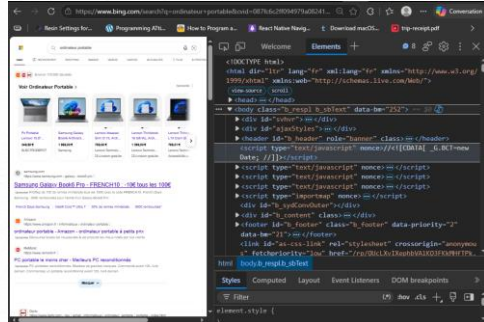
Architecture du WEB

- Le web se décompose en 2 ensembles:
 - Client / navigateur
 - Effectue des demandes vers un serveur
 - Requête http
 - Met en œuvre visuellement les structures
 - HTML / CSS
 - Exécute du Javascript pour manipuler la structure, gérer les events, ...
 - ECMAScript 262 => Javascript



Navigateur et fonctionnement

- Le navigateur affiche le DOM avec des représentation par défaut pour chaque élément
 - Document Object Model (représentation)
- Met en œuvre les propriétés css supplémentaire de chaque élément
- Fournis un moteur JS (v8) permettant l'exécution de fonctions pour manipuler la structure DOM et gérer les évènements UI
 - DOM (fonctions)
 - DOM Events
 - Fonctions API de js (fetch, ...)



Protocole HTTP / HTTPS

- Pour interagir avec les serveurs un protocole est utilisé :
 - » Hyper Text Transport Protocol
- Envoyé des requêtes
- Recevoir les réponses
- Il peut être surchargé d'un certificat pour garantir l'intégrité du contenu et de son émetteur:
 - SSL => HTTPS



Protocole HTTP / HTTPS

- Exemple de paquet HTTP :

```
HTTP1.0 /index.html GET
Corps POST *si nécessaire
```

- Composé de :

- Version de HTTP
 - 1.0, 1.1, 2.0
- Liens vers la ressource
- Méthode
 - GET, POST
 - PUT, PATCH, DELETE, OPTIONS (API REST)
 - BREW, PROPFIND, WHEN **pour Hyper Text Coffee Pot Control Protocol + code HTTP 418



Protocol HTTP / HTTPS

- Au delà du formalisme des demandes, il procure aussi dans les résultats de requêtes, un CODE HTTP de l'état de la réponse:

- OK

- 2XX (200)OK, (201)CREATED, ...
 - 3XX (301) redirection temporaire,(302) définitif, ...

- ERREUR

- 4XX (401) Unauthorized, (404) Not Found, ...
 - 5XX (500) Internal Server Error, (501) Not implemented yet,...

- Liste des codes exhaustifs selon la rfc :

https://fr.wikipedia.org/wiki/Liste_des_codes_HTTP

Les outils de développement

- Editeur de texte :

- Editeur pauvre

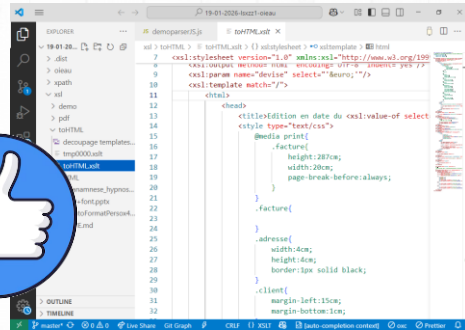
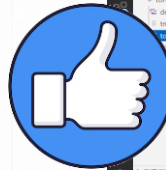
- Notepad / Vi / ...
- Notepad++, ...



- Editeur Riche

- Vs code

- Plugins
- Snippets
- WYSIWYG
- Complétion
- GIT, ...



Le Serveur de dev

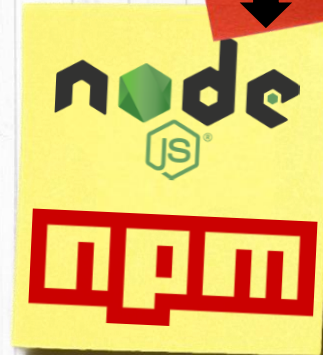
- Plusieurs possibilités :
 - Pour une appli FULL js seul un serveur type Apache, Nginx, IIS, ..., est nécessaire 
 - Node et Npm quand a lui :
 - Il fournit un gestionnaire de dépendance pour les paquets FRONT (bootstrap,...)
 - Un paquet serveur de fichier : lite-serveur
 - Un *mockeur* d API REST : json-server
 - Des outils de *lintage* (analyse de code configurable), *prettier* configurable, ... 

Node.js & npm

UTILISE DANS LE
COURS POUR LES
SERVEURS



- Node
 - Exécution de code JS coté serveur
 - Application
 - Scripting, ...
 - Lot d'applications existant
 - En ligne de commande
- Npm
 - Téléchargement de modules js
 - Gestion des dépendances

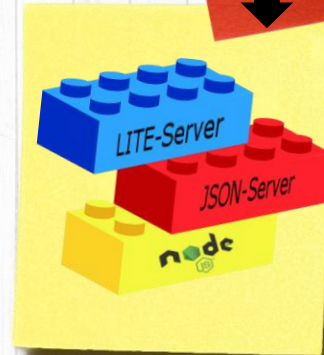


Serveur http & db

UTILISE
DANS LE
COURS *



- Lite-server
 - Server http
 - Browser sync (web socket) *port +1
 - Configurable
 - <https://github.com/johnpapa/lite-server>
 - npm install -g lite-server
- Json-server
 - Fake REST API
 - Stockage json
 - <https://github.com/typicode/json-server>
 - npm install -g json-server



***ou wamp**

JSON-server tuto : <https://codingthesmartway.com/create-a-rest-api-with-json-server/>

Serveur de version GIT

- Pour faciliter le maintient et le suivi du développement des applications
 - Historise les changements et les signes
 - Commit
 - » Suivi des changements
 - » Retour possible à des versions antérieures
 - checkout
 - Gere des branches de développements parallèles avec fusion possible
 - Pull Request



Github

- La plateforme privilégiée par les devs web
 - Git hub héberger sur internet
 - Gratuit
 - Sans limite de stockage
 - Commit : 50Mo+ / fichier =>Warning, 100Mo+ / Fichier interdit
 - Release : sans limite de taille
 - Repos public / privée
 - Possibilités collaboratives
 - VS code intégré dans les repos
 - Git lab hébergé par les entreprises



Serveur GIT

- Plateforme de disposition de code versionné

- Repository
- Gestion des commit distant
- Travail d'équipe



- Plateformes gratuites



- Github, GitLab
 - Hébergé ou officiel
 - 1 repo / account

- Atlassian BitBuckets *(Sélectionné pour le cours)
 - Multi-repository
 - Free (max 1Go/repo) ou payed account
 - Officiel uniquement



Git - bitbuckets

- Création d'un compte gratuit
 - <https://bitbucket.org/account/signup/>
 - <https://bitbucket.org/account/signin/>
- Création 1ere repository
 - Edit, commit, approuve du readme.md
 - Clone local de la repository
- Ouverture du répertoire sous Visual Studio Code
 - Edition du readme.md sous vscode
 - 1^{er} commit & push
- Approuve commit

Devtool & navigateur

- Le navigateur fait entièrement parti des devtools
 - Console embarquée
 - Log dans le code (`console.log`, `console.trace`, `console.table`, ...)
 - Exécution de code js a la volée
 - Gestion de sources
 - Breakpoint
 - Examen du stack d'appel
 - Watch
 - Map des code TS/js (cf. `tsconfig.json`)
 - Suivi de l'état de réseau
 - Manipulation des datas navigateur
 - `LocalStorage`, `SessionStorage`, cookies, ...
 - Mode mobile
 - Affichage
 - Vitesse réseau



Accessibilité Web

- Accessibilité et handicap
 - L'accessibilité = facilité d'accès au plus grand nombre ou facilité d'accès d'une entité à une fonctionnalité (vision qui ouvre à l'accessibilité des handicapés).
- WCAG, WAI et W3C
 - En IT, l'accessibilité est la faculté d'accéder à l'information et aux services à moindre coût, en offrant une interface accessible au plus grand nombre.
 - Le WAI (*Web Accessibility Initiative*) prône que :
 1. les outils de prod. de contenu doivent être accessibles à tous (ATAG)
 2. le contenu mis en ligne doit être accessible (WCAG)
 3. les outils de consultation (navigateurs) doivent être utilisables par tous
 - Nous sommes intéressés par le 2e point : l'accessibilité des sites Web.
 - Le WCAG 1.0 (*Web Content Accessibility Guidelines*)

Accessibilité et handicap : l'accessibilité est un terme général utilisé pour décrire le degré de facilité d'accès d'un produit (matériel, service, environnemental...) au plus grand nombre. Elle peut aussi être vue comme la facilité d'accès d'une entité à une fonctionnalité ou un bénéfice. Cette dernière approche de la définition ouvre le débat du droit individuel à accéder et tourne l'accessibilité vers les handicapés.

Accessibilité Web

- WCAG, WAI et W3C suite...
 - Niveaux de priorité du WCAG :
 - 1.A (must) : obligatoire
 - 2.AA (should) : souhaitable
 - 3.AAA (may) : optionnel
 - Un des critères de niveau A, par exemple, est le respect de HTML4.01 strict
 - Outils de reporting de conformité au WCAG (ex: Watchfire WebXACT)
(Remarque: plus disponible librement depuis le 1er février 2008)

24

24

Précisions :

A l'instar des notations boursières, le WCAG fournit 3 niveaux de l'accessibilité d'un site appelé niveaux de priorité et identifie :

1. l'indispensable (must) [A] : les concepteurs/réalisateurs de sites doivent s'y conformer sous peine d'interdire l'accès à certains groupes.
2. le souhaitable (should) [AA] : les développeurs devraient s'y conformer sous peine de rendre l'accès difficile à certains groupes (niv. mini USA).
3. l'utile (may) [AAA] : les développeurs peuvent s'y conformer pour faciliter l'accès de leur site à certains groupes de personnes

Un des critères de niveau A, par exemple, est le respect de HTML4.01 strict

Des outils tels que Watchfire WebXACT permettent de passer gratuitement un site au crible selon les normes WCAG et d'éditer un rapport. IBM a décidé d'en retirer l'accès public depuis le 1er février 2008 et l'a intégré à son logiciel « IBM Rational Policy Tester Accessibility Edition » (anciennement « WebXM Accessibility Module »).

Lois & accessibilité Web

- Accessibilité et gouvernements
 - Aux USA, via la Section 508 de la loi américaine Federal Rehabilitation Act (www.section508.gov), le gouvernement prône le développement web selon les normes minimales WAI-AA et la validation XHTML & CSS, l'insertion de touches raccourcis, une version hautement contrastée du site, la présence d'un média alternatif pour tout contenu (ex: texte + son).
 - En France, la loi 2005-102 du 11 février 2005 (art. 47) pour "l'égalité des droits et des chances, la participation et la citoyenneté des personnes handicapées", dite loi "handicap", rédigée par l'ADAE (Association pour le Développement de l'Administration Électronique, prévoit l'accessibilité physique généralisée (sans rupture de déplacement), l'accessibilité renforcée des ERP... ainsi que des sanctions et des dates butoirs pour les mises aux normes (phase de diagnostic en 2011 et échéance en 2015). (cf. <http://www.legifrance.gouv.fr>)

PREMIER PAS AVEC JS



Histoire de ECMAScript 262 (JS)

JavaScript aussi appelé **JS** est un langage de programmation de **scripts** utilisé dans les pages web principalement côté **client**.

JavaScript coté client est le seul langage natif dédié à la gestion de **l'interactivité des documents HTML**.

C'est un langage **orienté objet à prototype**, la base de l'objet est fournie par des objets équipés de **constructeurs** permettant de créer leurs propriétés.

JavaScript été créé en 1995 par Brendan Eich pour le compte de **Netscape Communications Corporation**. JScript dès 1996 dans MSIE 3.0 (développé par Brendan EICH de Netscape/Mozilla)

Le langage, actuellement à la **version 1.8.2** est une implémentation la norme **ECMA-262 apparue en 1997** et standardisée par l'**Ecma International**

JAVASCRIPT HTML DYNAMIQUE



Evolutions

- Il existe à ce jour 17 versions du standard ECMA-262
 - **ES3**, puis ES4 (abandonné pour ES3.1 nommé ES5)
 - **ES5**(2009) est la version la plus prise en charge aujourd'hui et qui répond toujours aux besoins d'aujourd'hui
 - **ES6** (ES2015) apporte de nouveaux concepts, *Promise, template String, arrow functions, module, destruct, ...*
 - Autrefois nécessitait une transpilation *es6->es5*
 - **ES7 (ES2016), ES2017, ES2018, ES2019, ..., ESNext**
 - Apporte tous les ans une nouvelle version avec de nouveaux apports techniques, nouvelles fonctions, ...
 - Peut nécessiter une transpilation pour compatibilité navigateur



Configuration environnement de dev

- **Projet Node**
 - Install Node.js (Nvm possible)
 - Projet Node
 - Install de paquets serveur
- **Git**
 - Install git scm
 - Configuration signature git
 - Création repos
 - Clone Repos
 - Premier commit
 - push
- **Vs code**
 - Plugins
 - Snippets

Install Node & NVM

- Nodejs
 - Installable disponible sur :
<https://nodejs.org/en/download>
- Node Version Manager
 - Permet d'installer n'importe la quelle des versions en une commande
 - » `nvm install xx.xx.xx`
 - Permet de passer d'une version a une autre en une commande
 - » `nvm use xx.xx.xx`
 - Installable nvm disponible sur :
 - » Windows
<https://github.com/nvm-windows/nvm>
 - » Mac/linux
<https://github.com/nvm-sh/nvm>

Node & npm

- Commandes npm

- Initialisation d'un répertoire de projet

- Créer un fichier minimal package.js

- » **npm init -y**

- Pré-configurer node avec le répertoire (si .git présent)

- -y : initialisation « quiet »



Il existe deux types de dépendances :
devDependencies
dependencies

- Installation des paquets

- » **npm install -g packetname packet2@version**

- -g : installation pour l'utilisateur courant,

- sans -g installation pour le projet, dans le répertoire Node modules

- --save permet l'ajout du paquet aux dépendances du projet

- Npm start vs npm run mon script

- Dans package.json, section scripts

- Script Name spécifiques nécessitant pas 'run' ex npm test, npm install, ...

Installation des dépendances Node

- devDependencies:
 - Serveur web :
 - npm install --save-dev lite-serveur
 - Serveur REST :
 - npm install --save-dev json-server
 - Typescript
 - Npm install --save-dev typescript
- dependencies:
 - Bootstrap
 - npm install bootstrap@latest

Github & git

- Installation de la prise en charge de git
<https://git-scm.com/>
- Création de compte gratuit sur
<https://github.com>
- Création d'une repositories sur github
- Clone de la repositories
 - Git clone <https://github.com/username/reposName.git>
- Configuration signature de commit
 - git config --global user.name "Your Name"
 - git config --global user.email "youremail@yourdomain.com"
- Premier commit
 - git add .
 - git commit -m message
- Push serveur
 - git puh

Plugin Vscode

- Linter
 - ESLint
 - Sonar
- Pretty
 - Prettier
- HTML/css
 - HTML CSS Support
 - Auto Complete Tag



Ou écrire du JS

- La balise script
`<script> console.log("hello");</script>`
- Les fichier js
`<script href="js/fichier.js"></script>`
- Dans les liens *
`<a href="JavaScript:alert('test');">Tester </a>`
- Dans les events **
`<input type="button" onclick="alert('test');" />`

*interdit par les règles d'accessibilités

** mauvaise pratique à éviter

Dans la mesure où les navigateurs comprennent plusieurs langages, il faut leur préciser quand ils doivent exécuter du JS. Pour cela, on place le code suivant :

« `<script type="text/JavaScript">...code JS...</script>` ».

Remarque: « `language="JavaScript"` est déprécié depuis HTML4.01 soit 1999

Conventions de nommage

- **Javascript, sensible à la casse, utilise :**
 - La **UpperCamelCase** (casse de chameau événement) pour les types / namespaces
 - Ex: RegExp ou String - Node ou Ajax
 - La **lowerCamelCase** pour les objets, variables, propriétés, fonctions, méthodes
 - Ex: window.clientHeight ou xhr.readyState - charAt(2) ou indexOf('?')
 - Tout en **majuscule**, chaque mot séparé par un « _ » pour les **constantes**
 - Ex: Node.TEXT_NODE ou STR_ERROR_MSG
 - Tout en **minuscule** pour les **événements** et gestionnaires d'événement
 - Ex: onreadystatechange ou onclick
 - « _ » **devant** le nom des propriétés **privées** (ex: _myPvtProp)
 - Ex: var _id = 0
 - un **verbe** en début de méthode et is/has/can pour celles **booléennes**
 - Ex: hasChild() ou isEmpty()



règles syntaxiques

- ; Point virgule signifie la fin d'une instruction
- . Point sert à accéder aux méthodes d'un objet
- () Parenthèses pour les appels et passages d'argument à une fonction
- [] Crochets pour accéder à l'élément d'un tableau
- {} Accolades pour délimiter un bloc d'instructions
- " Apostrophe * & "" guillemets pour délimiter les zones de texte

* conseillé

Valeurs particulières

- NaN → Not a Number , calcul impossible sur des valeur pas numérique ou division par 0
- null → Valeur non instancié
- ***undefined*** → méthode ou propriété inexistante
- "" ou " → chaine vide

Variables

Une **variable** est un espace de **stockage nommé**. La **déclaration** de cet espace permet d'y **affecter** une valeur accessible par le nom de la variable.

Pour créer une variable on utilise le mot réservé "**var**", suivi du nom de la variable et/ou d'une valeur par défaut (**new** pour construire un type de données)

- **var** strMsg = 'message à afficher !';
- **var** strMsg = **new** String('message à afficher');
- **var** strMsg = **new** String;
strMsg = 'message à afficher';

Remarque: le mot réservé "var" n'est pas obligatoire, il permet seulement de limiter l'existence de la variable au bloc de code dans lequel elle a été créée.

Ceci évite les problèmes de collision avec d'autres variables du même nom créées dans d'autres blocs de code.

JAVASCRIPT HTML DYNAMIQUE

PRECISIONS

- Pour ceux qui pratiquent des langages tels que le C, JS étant souvent interprété (cf. linker), ne séparez pas définitions, déclarations et init.

- On déclare une variable par : « **var** variable = valeur (typée); » ou « **var** variable = **new** Classe(); » ex: var intNum = 5; var strTmp = new String();

Remarque: faites attention aux problèmes de linkage avec this et son utilisation combinée avec new, les méthodes (anonymes ou non), les gestionnaires d'événements...

- Pour les fonctions/méthodes on met le terme « **function** » suivi du nom de la fonction et de parenthèses contenant les paramètres à transmettre.

cf: **function** fonction(param1, param2...){...code...}

ex: **function** check(prmStrTxt){ var intPos=prmStrTxt.indexOf("?");...}

- La zone entre parenthèses est réservée à la transmission de paramètres (vide si rien) (nb libre par rapport à la définition qui ne sert qu'à nommer les params). Ex: **function** validForm() { document.f.submit(); }

- La valeur de retour est retournée par la fonction « **return**(valeur); » simplifiée en « **return** valeur; » pour compatibilité avec les anciens codes.

Types

- Types de données
 - numérique
 - **Number** => **123**
 - Etat booléen
 - **Boolean** => **true**
 - Chaîne de caractères
 - **String** => **"abc"**
 - Tableaux
 - **Array** => **[1,2,3]**,
 - Objet
 - **Object** => **{a:1,b:'e',}**
 - Instance d'objet spécifique
 - **Date** => **new Date()**
 - Null
 - *Valeur instanciée sans valeur* => **null**
 - Non défini
 - *absence de valeur* => **undefined**

ATTENTION

LES VALEURS
DÉCIMALES SONT
SIGNIFIÉ PAR

• (POINT)

ET NON PAR

(VIRGULE)

Préférez les types natifs plutôt que les types objets

ex

```
var str='blabla' plutôt que var str=new String('blabla');
```

```
var arr=[] plutôt que var arr= new Array();
```

typeof

- Détection de type d'une variable
 - typeof

```
console.log(typeof 42);  
// expected output: "number"  
  
console.log(typeof 'blubber');  
// expected output: "string"  
  
console.log(typeof true);  
// expected output: "boolean"  
  
console.log(typeof  
declaredButUndefinedVariable);  
// expected output: "undefined";
```

Transtypage

- Conversion de type

- Chaîne vers numérique

```
Number.parseFloat(maStringVar);  
//conversion vers décimal  
Number.parseInt(maStringVar);  
//conversion vers entier
```

- Chaîne vers Date

```
var date1 = new Date('December 17, 1995 03:24:00');  
// Sun Dec 17 1995 03:24:00 GMT...  
  
var date2 = new Date('1995-12-17T03:24:00');  
// Sun Dec 17 1995 03:24:00 GMT..
```

Opérateurs Logiques & Arithmétiques

- Opérateurs arithmétiques
 - $+$, $-$ → addition, soustraction
 - $++$, $--$ → incrémentation ou décrémentation
 - $*$, $/$ → multiplication, division
 - mod*** → modulo (reste de division euclidienne)
- Opérateurs logiques
 - &&***, ***||*** → et, ou
 - <***, ***>***, ***<=***, ***>=*** → sup, inf, inf ou égal, sup ou égal
 - !*** → NON booleen
 - ==***, ***!=*** → strictement égal, différent
 - ===***, ***!==*** → égal en valeur et en type, différent en valeur et en type

Array

- Initialisation groupée :
var prenom = ['Alex', 'Pascal', 'Laurent'];
- Lire ou modifier par l'index :
prenom[0] = 'Nikola';
- Taille d'array
prenom.length
- Sélectionner :
prenom.slice(idxDbt[, idxFin])
- Parcours avec for in
for(i in prenom){...code...}
- Parcours avec Fonction de l'instance d'array
array.forEach((element,index,liste)=>{...code...})

Array

- Listage

prenoms.join(';');

– (join() sep = « , ») et toString() (idem join())

- Tri

prenoms.sort(); ou

numbers.sort(function(a,b){return a - b})

- Concaténation de tableaux

prenoms.concat(prenoms2);

Array

- Supprimer des éléments :
 - pop()
 - le dernier et le renvoie,
 - shift()
 - le premier
- Ajouter des éléments :
 - push(a,b,...)
 - à la fin
 - unshift(a,b,...)
 - au début
 - splice(idx, nbr, emt1, emt2...)
 - à partir de l'index
- Inverser l'ordre : reverse()

Afficher le code : toSource() presque en notation JSON (pas sous MSIE6-)



PROGRAMMATION JAVASCRIPT FONDAMENTALE



Scoping / portée de variable

- ES5
 - **var**
 - Disponible dans tous les blocs de code enfants
 - Global si déclarer en dehors de toute fonction
 - » Instancie la variable dans l'objet *window*
 - Disponible dans toute le contexte javascript
 - » Ex: `var x='une variable'`
- ES6
 - **let et const**
 - *sont disponible dans tous les enfants* du contexte de déclaration (fonctions enfants)
 - Remplace l'usage de *var*
 - **let**
 - Variable donc la référence peut être remplacée par une nouvelle valeur
 - » Ex: `let uneVariable='abc'`
 - **const**
 - Constante dont la référence de la valeur ne peut pas être remplacer par une autre
 - Doit être affectée lors de la déclaration
 - » Ex: `const uneConst='abc'`
- ES6 & Scope module
 - Lors d'un import `<script type="module" src="...">` ou `import`
 - Les `let` et `const` déclarer en dehors de toute fonction, on pour accessibilité seulement le fichier qui la contient
 - Notion d'import / export

La gestion de imports

- Avec import et **export / default / import {} from**

- Syntaxe :

- pour être importer, un contenu doit être exporter

```
export class ExportedThing{};  
export default class DefaultExported{...};
```

- Puis importer a la demande

```
import {ExportedThing} from './rep/fichier';  
import DefaultExported from './rep/fichier';
```



Renommer un import avec as

Import { exportedVar as myNewVarName } from './fichier'

Il existe d'autres patterns pour la construction de module
Ces autres patterns sont pris en charge par web pack

ex: pattern amd et la fonction **require**('...');

Les test if/else if / else

- si condition

if (condition){instruction;}

- sinon si condition

else if (condition){instruction;}

- sinon

else {instruction;}

- ternaire

(condition)? SiVrai: SiFaux;

les test switch/ case / default

- selon cas

```
switch (variable)
{
    case value :    instruction;
                    break;
    case value :    instruction;
                    break;
    default: instruction; break;
}
```

*le cas **default** est une bonne pratique

Les Boucles

- Tant que

- Le nombre de tours est inconnu
- La condition est vérifiée au début de chaque tours
- Exécution de contenu pas obligatoire

```
while(maValeur != 10 )  
{  
    maValeur=instruction();  
}
```

Les Boucles

- Faire ... tant que
 - Le nombre de tours est inconnu
 - La condition est vérifiée a la fin de chaque tour
 - Exécution de contenu obligatoire au moins un fois

```
do{  
    maValeur=instruction();  
}while(maValeur != 10 );
```

ATTENTION
UN ; (POINT VIRGULE)
EST NÉCESSAIRE
APRÈS LA FIN DE LA
CONDITION DU
WHILE



Les Boucles

- Pour

- Le nombre de tours est connu
- un l'itérateur est instancié au départ
- l'itérateur est incrémenté a chaque tour
- La condition est vérifiée a la fin de chaque tours
- Exécution de contenu obligatoire

```
for(var i=0; i<10;i++)  
{  
    instruction();  
}
```

Fonctions

- Déclaration de fonction

- Valeur de retour facultatif
- arguments d'entrée facultatif

```
function nomFonction(argument1, argument2)
{
    instruction(argument1);
    var uneValeur= argument1 + argument2;
    return uneValeur;
}
var nomFonction2= nomFonction;
```

- Appelle de fonction

- Affectation du retour et variable d'entrée facultatif

```
var z=nomFonction(123, uneValeur);
nomFonction2(123, uneValeur);
```

Fonctions

- Les arguments d'entrées non déclarées
 - un tableau des variables d'entrées auto déclarée
 - si pas d'argument d'entrée déclarée lors de la déclaration de la fonction
 - Mot clef : ***arguments***

```
function nomFonction(){  
    console.log(arguments[0]);  
}  
nomFonction(123);
```

Gestion des exceptions

En JS on peut aussi gérer les erreurs avec **try** (essaye), **catch** (attrape l'erreur), **finally** (finalement fait) et **throw** (renvoi l'erreur).

Les exceptions prédéfinies sont
Eval/Range/Reference/Syntax/Type/URI/InternalServerError.

```
try
{
    ... code qu'on essaye d'exécuter
}
catch(e)
{
    ... code d'autres erreurs
} finally
{
    ... exécuté obligatoirement
}
```

JAVASCRIPT HTML DYNAMIQUE



With

- Simplification avec with quand on veut faire plusieurs opérations sur un objet, on peut utiliser with

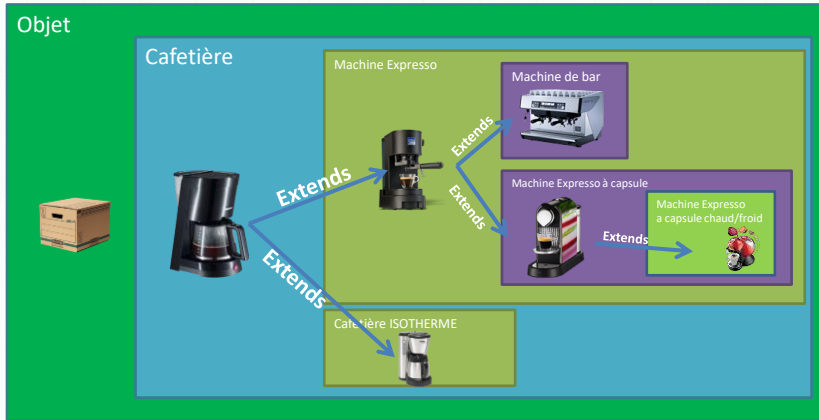
```
with ( objTxt.style) {  
    border = '1px solid black';  
    fontSize = '12px';  
    fontWeight = 'bold';  
}
```

INTRODUCTION À LA PROGRAMMATION OBJET EN JS



Notion d'objet

Objet



Notions d'objet

L'écosystème Objet, un programme peut être conçu comme un écosystème rempli d'entités en interaction (**objets**), disposant de caractéristiques propres (**propriétés**) et de moyens d'action (**méthodes**) et répondant à des stimuli externes (**événements**).

On appelle "objet" une instance d'un modèle d'objet (classe*)

Ex: une poule marron avec une seule patte (un objet) fait partie de la famille des poules (sa classe)

On appelle "propriété" une variable et "méthode" une fonction de la classe*

Ex: la couleur de la poule est une de ses propriétés. Le fait qu'elle plane est une méthode.

On appelle gestionnaire d'événement une fonction qui répond à un stimuli

Ex: quand un renard entre dans le poulailler, les poules piaillent et s'enfuient



Notions d'objet

En JavaScript, tout ceci donne :

```
var myCutsom = new Custom({prop:1});  
  
myCutsom .couleur = 'marron';  
  
myCutsom .plane();  
  
myCutsom.onCustomEvent= function() { ... }
```



Notions d'objet

Il existe des mots clés réservés à la manipulation des objets.

this

- this = objet courant, à savoir:
niveau global : objet window courant (frame)
fonction / méthode : l'objet associé à la méthode
événements : objet déclencheur.

new et delete

- opérateur new pour créer (allouer) un nouvel objet.
- chaîne non-allouée et null
- pour dés-allouer utiliser delete (à noter : JS ne fournit pas de gestionnaire de destructeur car il dispose d'un ramasse-miettes = « garbage collector »)

JAVASCRIPT HTML DYNAMIQUE



this

- this est un mot réservé qui fait référence à l'objet courant, c'est à dire : au niveau global, l'objet window courant (frame), dans une fonction (méthode), l'objet associé à la méthode (peut-être window pour les fonctions globales), dans les événements c'est l'objet déclencheur.

new et delete

- l'opérateur new est un opérateur spécial qui permet de créer (d'allouer) un nouvel espace mémoire pour l'objet.

- une chaîne pour laquelle aucun espace mémoire n'est allouée est null

- pour dés-allouer la mémoire qui a été réservée, il faut utiliser l'opérateur delete. C'est important si on manipule de grandes quantités de données (ex: Array de grande taille).

Notions d'objet

Des objets ou des classes ?

- Si on peut étendre les types existants à l'aide de la méthode prototype, les langages de prototype ne permettent pas de créer des classes.
- On peut néanmoins simuler les classes et espaces de noms en utilisant des conteneurs comme suit : `var Ajax = {}... Ajax.Updater = { ... code ... }`
- Et simuler des classes en combinant conteneurs, la portée des variables et méthodes, l'usage de la méthode prototype...
- NB: utilisez des fonctions anonymes pas `new Function` sinon il risque d'y avoir des problèmes de lien.



Notions d'objet

Implémentation de classes

- les propriétés privées sont déclarées dans l'objet avec 'var' on ne peut y accéder que par des méthodes privées ou privilégiées.
- les méthodes privées déclarées inline dans le constructeur ou par : « `var functionName = function(){...}` » (même portée que précédent).
- les méthodes privilégiées déclarées par : « `this.methodName=function(){...}` » et invoquées depuis l'extérieur de l'objet.
- les propriétés publiques déclarées par « `this.variableName` » et lues et modifiées depuis l'extérieur de l'objet.
- les méthodes publiques sont définies par « `Classname.prototype.methodName = function(){...}` » et appelées depuis l'extérieur de l'objet.
- les propriétés de prototype par : « `Classname.prototype.propName = value` »
- les propriétés statiques définies et utilisées par « `Classname.propName` »
- **JavaScript ne supporte pas les variables et méthodes protégées !**



Notions d'objet

Simuler l'héritage

- Une classe peut hériter par « `ChildClassName.prototype = new ParentClass();` »
- Il faut penser à rétablir immédiatement le constructeur, par :
« `ChildClassName.prototype.constructor = ChildClassName;` »
- On peut appeler les méthodes des ancêtres (pas leurs propriétés privées) que vous avez surchargées à l'aide de la méthode `Function.call()`, ex :
« `Chat.prototype.dors = function(){ Mammifere.prototype.dors.call(this);...}` »
- Pour garder une trace du parent (afin de ne pas avoir à se souvenir de tous les parents / enfants) on peut créer une propriété privée « parent », ex :
« `Chat.prototype.parent = Mammifere.prototype;... this.parent.dors.call(this);` »
- On peut simplifier la notation en créant une fonction par prototypage sur le type `Function` (ex: `heriteDe`) et en instanciant les objets comme des fonctions.



Class

- Le mot clef **class** est vraiment mis en œuvre dès l' ES6 en es5 une fonction peut devenir un objet dès lors qu'il est instancié avec le mot clef new
- Pour rendre un champ disponible publiquement, il doit être défini sur l'instance **this** dans la fonction, il sera accessible que préfixé du mot clef **this.**, dans la fonction

```
function ObjetByFunc(param){  
  this.value=2  
  //variable public  
  var privateValue=2  
  //variable local  
  function fn1(){  
    //fonction locale  
    privateValue=5  
    this.value=5  
  }  
  this.publicFn=fn1;  
  //exposition public d'une fonction  
}  
var instance=new  
ObjetByFunc('value');  
instance.value=5;
```

Principaux objets

Fonctions/Classes globales (native ou built-in)

- built-in = intégrées nativement (ex: **Boolean**, **Number**, **String**, **undefined**...). Accessibles partout et sans propriétaire.

Les chaînes de caractères (String)

Quelques propriétés et méthodes des chaînes de caractères:

- **String()** son constructeur, ex: « **var strTxt = new String();** »
- **indexOf** position d'une sous-chaîne ex: « **strTxt.indexOf('?');** »
- **charAt()** le caractère situé à une position (index) ex:
« **strTxt.charAt(3);** »
- **length** propriété indiquant la longueur de la chaîne

JAVASCRIPT HTML DYNAMIQUE

Fonctions/Classes globales (native ou built-in)

- Certains objets, classes et fonctions sont built-in, c'est à dire intégrées nativement à JS. C'est le cas de Boolean, Number, String, undefined... Les fonctions built-in sont accessibles partout et n'ont pas de propriétaire.

- Les fonctions parseInt et parseFloat sont Built-in. « parseInt » renvoie un entier sur la base d'une chaîne (interprétée) et d'une base mathématique (ex: base 10 ou base 16). Ex: « parseInt('152.25', 10); » renvoie « 152 » et « parseInt('0xA1'); » renvoie « 161 ».

Remarques: les chaînes commençant par « 0x » sont interprétées en base 16 par défaut, celles commençant par 0 comme de l'octal (d'où certains déboires parfois avec les dates), les autres en base 10.

Principaux objets

La classe statique Math

La classe Math est particulière et utile pour des calculs, ex:

- **Math.floor(valeur)** renvoie l'entier inférieur (pas la partie entière !)
- **Math.random()** renvoie un chiffre à virgule aléatoire entre 0 et 1

Gestion du temps (Date)

Quelques informations, propriétés et méthodes pour bien gérer les dates:

- Rappels sur les dates (GMT, US vs FR, Julian, DateStamp...)
- **Date()** constructeur, ex: « **datTmp = new Date();** » (date affectée ici)
- les méthodes **set/getTime(tps en sec)** permettent de lire et de fixer l'heure
- la méthode **toGMTString()** convertit une heure en heure GMT (décalage)



Principaux objets

70

screen, window, navigator

- **screen** = écran de l'internaute (réso: width / height)
- **navigator** = navigateur de l'internaute (MSIE, MFF), (info de param)
- **window** = fenêtre courante (frame si frameset)

document

- **document** = page actuelle (write, clientWidth/Height, arbre DOM)

e-mails (« mailto » pas js mais assimilée)

- mailto commande HTML assimilée JS pour ouvrir le mailleur paramètres : adresse, subject, body...
- tel, sms fournit par html5

PRECISIONS

- L'objet screen correspond à l'écran de l'internaute, on peut par exemple récupérer screen.width, screen.height pour connaître sa résolution

- L'objet navigator correspond au navigateur de l'internaute (MSIE, MFF), on peut récupérer des informations de paramétrage afin de personnaliser (si besoin) le code. Ex:

« navigator.javaEnabled(); » permet de savoir s'il y a une JVM

« navigator.appName.indexOf("Netscape"); » pour ce type de navigateur

- L'objet window correspond à la fenêtre courante (frame si frameset), on peut par exemple récupérer l'URL (pratique pour la méthode GET),
ex: « window.location » et « window.location.search »
[window].alert('message'); pour afficher un message dans une dialogbox
[window].history.go(-1); pour revenir 1 action en arrière
[window].close(); pour fermer la fenêtre courante
window.open(URL,nom_frame,params_frame) (précis & sans espaces)

document

- L'objet document correspond à la page chargée, on peut utiliser :
document.write('texte à écrire') pour écrire dans la page
document.body.clientWidth (ou Height) pour savoir la taille intérieure de page
document.forms[0], document.layers['lyr1'] pour accéder aux objets (DOM0)

- Par exemple, avec un script simple on peut procéder à des permutations d'images. « documents.images[0].src = ».

e-mails (« mailto » pas js mais assimilée)

- La commande mailto est une commande HTML souvent assimilée à du JS, elle permet d'ouvrir l'outil d'envoi d'e-mails (MS OE, Mozilla Thunder Bird...) de l'internaute en lui transmettant des paramètres (sujet, adresse e-mail du destinataire, texte par défaut ...). On la met dans un lien.

- ex: « mailto:info@web.com?subject=Sujet%20du%20courrier&body=... »

Le JSON

- Le JSON Syntaxe proche de js
 - Basé sur un sous-ensemble ECMA-262 (3e éd.)
- Conversion rapide de JSON à JS
- l'objet javascript *JSON*
 - *JSON.parse(chaine)*
 - une chaine json vers js
 - *JSON.stringify(object)*
 - un Objet js vers chaine json



Le JSON

- La syntaxe JSON (JavaScript Object Notation)

- Un array (tableau) = [value, value, value]

- Ex: [1, 2, 3] ou ["Albert", "Einstein", 1960]

- Un object (objet) =

- { "propriete":value, "propriete":value }

- Ex: { "prenom": "Albert", "nom": "EINSTEIN", "age": 64, "adresse": { "cp": 75008, "ville": "paris" }, "enfants": [{ "prenom": "José" }, { "prenom": "Bob" }] }

- Les paramètres de hash : idem au JSON mais sans les " autour des propriétés des objets, ex :

- { prenom:"Albert", nom:"EINSTEIN", age:64, adresse:{ cp:75008, ville:"paris" }, enfants:[{ prenom:"José" }, { prenom:"Bob" }] }



Le JSON



- La syntaxe JSON (JavaScript Object Notation) :
 - Une value (valeur) = number, string, object, array, null, true, false
 - Un number (nombre) = chiffres et nombres entiers / décimaux / exponentiels
 - Ex: 12 ou 12.51 ou 0.221 ou 5e-6 ou -15.50
 - Un string (chaîne) = "chaîne classique + caractères d'échappements (\n, \\...)"
 - Ex: "nom", "petite phrase \nsur plusieurs lignes"



JSON

- JSON → Javascript Object Notation
 - Un format concurrent à XML
 - JSON est un langage structuré basé sur la syntaxe Javascript
 - Association clefs / valeur
 - Différents types de valeurs :
 - Object, array, number, string, null
 - JSON possède désormais une technique de validation
 - JSON Schema
 - intégrable en js jusqu'à la version 2019



JSON

UN FLUX EST
TOUJOURS
CONTENU ENTRE
{...}
PRINCIPALES

Code JSON - geojson

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [
          102.0, 0.5
        ]
      },
      "properties": {
        "prop0": "value0"
      }
    }, { "type": "Feature", [...] }
  ]
}
```

Représentation

- Association

- "Clef" Toujours entre " "
- Valeur Plusieurs types

- Array

- [items]

- Number

- Objet

- { contenu }

- String

- "Valeur de la chaîne"

- Séparateur de champs → ','

JSON et S rialisation / D s rialisation

- La s rialisation est un process qui permet de rendre une instance d'objet enregistrable

- Seules les valeurs sont pr serv es
- Aucune fonction n'est gard e

JSON.stringify(monObjet)

- La d s rialisation, est le processus inverse, il cr e une instance d'objet a partir des valeurs

JSON.parse('cha ne JSON')

- **Attention** la d s rialisation rend une instance d'objet g n rique. Si c'est pour peupler une instance d'objet sp cifique et que les valeurs correspondent alors on peut merger les objets

Object.assign(instanceClass,objectParsed)

- Si les champs ne correspondent pas, une fonction de mapping du parsing g n rique devra  tre  crite



6. EVÈNEMENTS JS

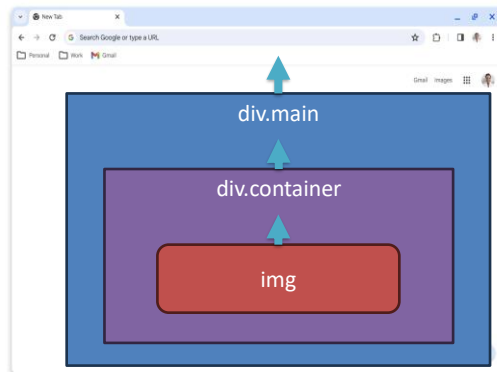


System d'évènements

- La représentation DOM est l'affichage des balises, pour interagir il nécessite la gestion des évènements qui se produisent dans la fenêtre
 - Chaque action déclenche un évènement associé au type d'action rencontré
 - DOM EVENT
 - MouseEvent
 - KeyboardEvent
 - Ces événements peuvent déclencher des fonctions associées lorsqu'elles sont écoutées
 - `baliseDOM.addEventListener('event',{objtEvent}=>{.....})`
- Les évènements sont liés à un contexte où ils sont écoutés
 - Balise
 - Window
 - Storage, ...

La propagation des events

- Les events sont par nature propagés :
- Lorsque on clique sur l'image, indirectement on clique sur :
 - Div.container
 - Div.main
 - Window
- Chaque niveau de balise peut posséder sa gestion de l'évènement *click* si il est Disponible dans la balise
- L'évènement est donc diffusé à chaque couche de balises parent contenant la balise déclencheur de l'évènement
- Il est possible d'arrêter la propagation :
 - ***objetEvent.stopPropagation()***



Comportement par défaut d'événement

- Certaines balises possèdent un comportement par défaut lors du déclenchement de certains événements :
 - Liens `<a href=« »>`
 - rechargement de la page vers *href*
 - Formulaire `<form action=« » method=« »>`
 - Rechargement de la page vers le lien *action*
 - Surcharge de l'URL avec query string si *method="GET"*
- *Ces comportements peuvent être arrêtés par l'objet d'événement grâce à **objetEvent.preventDefault()***
 - Liens sur l'événement *onclick*
 - Formulaire sur l'événement *onsubmit*



Event et formulaire

- Les formulaires possèdent 2 events spécifiques
 - **submit**
 - Évènement levé lorsque un *Button* type *submit* est pressé ou par instruction js
 - Cette action doit en premier stopper le comportement par défaut du formulaire (*preventDefault*) avant toute autre instruction
 - Cet évènement peut être déclenché par js : `monForm.submit()`
 - **reset**
 - Évènement levé lorsque un *Button* type *reset* est pressé ou par instruction js
 - Cette action remet tous les *inputs* à leurs valeurs émises dans l'attribut *value*
 - Cet évènement peut être déclenché par js : `monForm.reset()`
- Pour rappel, les inputs sont accessibles depuis la balise formulaire par leurs *name* ou position:
 - `monForm['inputName'].value` / `monForm[0].value`



Evènement souris

- Cette catégorie permet la gestion des events lié a la souris.
 - Plusieurs types d'events, tous issues de mouse event
 - `dragEvent`
 - `wheelEvent`
 - `PointerEvent`
- Principaux events :
 - **`mouseenter/mouseover/mouseleave`** : entrer, survole, sortie du pointer
 - **`mouseDown`** : quand un bouton est enfoncé
 - **`click`** : un bouton est enfoncé puis relâché
 - **`mouseUp`** : un bouton est relâché
 - **`dblclick`** : lors d'un double click
- Il existe des event spécifique a la roulette et aux action *drag and drop*:
 - **`dragStart`**
 - **`dragEnd`**



Keyboard event

- Les évènements du clavier
 - Type : KeyboardEvent
 - keydown : lorsque une touche est enfoncée
 - input : lorsque une touche est enfoncé puis relâchée
 - keyup : lorsque une touche est relâchée
 - focusin/focus/focusout : lors des focus des sur les inputs
- Cet *event* se déclenche lorsque une action au clavier est faite



L'événement online

- Connaître l'état de connexion au réseau

- Etat de la carte pas état de la connexion

📄 Online.js

`window.navigator.onLine` ou `navigator.onLine`

- Un event permet de gérer le changement d'état

📄 changeState.js

```
window.addEventListener('offline', (e) => { ... })
```

```
window.addEventListener('online', (e) => { ... })
```

Liste des Type d'événements

Voici tous les types d'événements disponibles avec leurs spécificités. Tous ces types d'événements héritent tous de l'objet Event

- AnimationEvent
- AudioProcessingEvent Deprecated
- BeforeUnloadEvent
- BlobEvent
- ClipboardChangeEvent
- ClipboardEvent
- CloseEvent
- CompositionEvent
- CustomEvent
- DeviceMotionEvent
- DeviceOrientationEvent
- DragEvent
- ErrorEvent
- FetchEvent
- FocusEvent
- FontFaceSetLoadEvent
- FormDataEvent
- GamepadEvent
- HashChangeEvent
- HIDInputReportEvent
- IDBVersionChangeEvent
- InputEvent

- **KeyboardEvent**
- MediaStreamEvent Deprecated
- MessageEvent
- **MouseEvent**
- MutationEvent Deprecated
- OfflineAudioCompletionEvent
- PageTransitionEvent
- PaymentRequestUpdateEvent
- **PointerEvent**
- PopStateEvent
- ProgressEvent
- RTCDataChannelEvent
- RTCPeerConnectionIceEvent
- StorageEvent
- SubmitEvent
- TimeEvent
- TouchEvent
- TrackEvent
- TransitionEvent
- UIEvent
- WebGLContextEvent
- WheelEvent

7.MANIPULATION DOM



Fonctions du DOM

- Le DOM c'est aussi un ensemble de fonctions JS pour la manipulation du DOM:
 - Création d'éléments
 - Navigation dans les éléments
 - Ajout suppression de balise
 - Manipulation des attributs des éléments



Sélection des éléments du DOM

- Pour sélectionner un élément plusieurs possibilités depuis un document (ou fragment) DOM instancié:
 - Sélection balise unique
 - Renvoie l'élément DOM unique
 - `document.getElementById('id')`
 - Sélection liste de éléments
 - Revoit une *HTMLCollection* pas *Array*,
 - Attention pas hérité de *Array* donc pas `.forEach()`, `.map()`, `.find()`, ...
 - `document.getElementsByTagName('baliseName')`
 - `document.getElementsByTagNameNS('baliseName')`
 - `document.getElementsByClassName('className')`
 - `document.getElementsByName('attribNameValue')`

Sélection avancée par Sélecteur CSS

- Pour faciliter la sélection, une série de fonction permettent la sélection par sélecteur css :
 - Sélection unique du 1^{er} élément trouvé
 - Renvoi l'élément unique
 - **`document.querySelector('cssSelector')`**
 - Sélection multiple
 - Renvoi *NodeList* d' élément(s) trouvé(s)
 - Hérité de *Array* donc présence de **`.forEach()`**, **`.map()`**, **`.find()`**,...
 - **`Document.querySelectorAll('cssSelector')`**



Sélecteur Css

- **Eléments**

```
p { font-size: 90%; margin: 0.5em; padding: 2px; }
```
- **Classe**

```
.important { color: red; }
```
- **Identifiant**

```
#menuGauche { margin: 0; padding: 0.5em; }
```
- **Pseudo-classe**

```
a:hover {  
border-top : 1px solid black;  
border-bottom: 1px solid black; }
```

90

Élément : un élément correspond à une balise (X)HTML telle qu'on la trouvera dans la structure du document, notée sans <> (h1, p, a). On peut ainsi définir des instructions CSS pour des éléments et façonner un document de base.

Classe : lors de sa définition, une classe est identifiée par un point (.) précédant son nom (.maClasse). Vous pouvez choisir le nom que vous voulez, sans espaces. Elle pourra être utilisée sur différents éléments (X)HTML en leur ajoutant l'attribut class et la valeur de la classe entre guillemets sans le point (class="maClasse").

Identifiant : un identifiant fonctionne comme une classe. Il est référencé par un dièse (#) avant le nom. Un identifiant sert pour un élément (X)HTML unique en se basant sur l'attribut id des balises.

Pseudo-classe : les pseudo-classes permettent de mettre en forme des parties intégrantes du DOM. Il en existe peu :

- :active (élément actif),
- :focus (élément en focus),
- :hover (élément survolé),
- :link (non visité),
- :visited (élément visité),
- :first-child (premier élément fils),
- :lang (permet de spécifier une langue)

CSS 3.0 sélecteurs

- Sélecteur nouvelle génération

- Adjacents

```
ul + p { color: red; }
```

<p> directement précédé de

- Enfants

```
div#container > ul { border: 1px solid black; }
```

Uniquement les enfants contenu dans div#container

- Présence d'attribut

```
a[title] { color: green; }
```

Lien comportant un attribut 'title'

- Valeur d'attribut

```
a[title="greenLink"] { color: green; }
```

Lien comprenant l'attribut 'title' avec 'greenLink' comme valeur

Et plein d'autres : <http://code.tutsplus.com/fr/tutorials/the-30-css-selectors-you-must-memorize--net-16048>

Rappel Sélecteur css

- Syntaxe de Base
 - sélecteur { propriété: valeur; }
- Sélecteurs multiples

```
Code css :  
h1, h2, ... h6 {  
    font-family : "MS Trebuchet", Verdana, Arial, sans-serif;  
    font-weight : bold;  
}
```

- Encapsulation des éléments

```
/* s'applique aux images contenues dans un lien*/  
a img { border : none; }  
/* s'applique aux paragraphes dans les balises class maClasse*/  
.maClasse p { margin : 0.5em 0; }
```

CSS Histoire et normes

- Les feuilles de styles (cf. éditeurs de textes évolués) ont été inventées dans les années 70 au début de la mise en place du SGML, mais ce n'est qu'en 1994 que la norme CSS1 telle qu'on la connaît est apparue.
- La norme CSS2 est apparue en 97 afin d'intégrer de nouvelles fonctionnalités (ombres, direction, positionnement) mais suite à des problèmes d'intégration sur les navigateurs, elle a été transformée (plus stricte) en CSS2.1 en 2005. Aujourd'hui aucun navigateur n'est encore totalement compatible avec la norme CSS2 (même pas Firefox).
- La norme CSS3 débutée en 98 est toujours en cours de développement, elle prévoit la modularisation (cf. XHTML1.1)

Précisions : Il est possible et même recommandé d'aérer le code css. CSS permet une forme de type bloc utilisé dans beaucoup de langages de programmation. En effet des délimiteurs de bloc ({}) autorisent la formulation d'une instruction par ligne.

Sélecteurs multiples : On peut déclarer des instructions CSS pour des sélecteurs multiples. Il suffit de séparer ses sélecteurs par des virgules.

Encapsulation des éléments : On note l'encapsulation des éléments, on les classe en les séparant par un espace. Par exemple: a img signifie que le bloc de propriétés s'appliquera aux images contenues dans un lien.

Création d'un élément DOM

- Le DOM doit créer l'élément, sont contenu de balise et ces attributs pour permettre son ajout dans le dom.
- Contrairement a innerHTML dans un parent, ajouter ce nœud par fonction DOM ne détruira pas le DOM du parent (events...)
- Pour préserver le contexte de mise en œuvre du nœud, le document ciblé doit être l'objet de création du nœud
 - **`const elem=document.CreateElement('nomDeBalise')`**
 - **`const elem=document.CreateElementNS('nomDeBalise','http://www.w3c.org...')`**
- Il est possible de créer aussi des attributs pour ajouter à cet élément ou autre élément :
 - **`const att=document.createAttribute('attrName')`**
 - **`const att=document.createAttributeNS('attrName','NS URI')`**
 - **`att.value='value'`**



Manipulation DOM

- Manipuler le contenu de balise
 - Les fonction inner (getter / setter)
 - **`element.innerHTML='nouveau contenu'`**
 - » Il est possible de donner a cette chaine un contenu textuel contenant aussi des balises
 - » La fonction innerHTML détruit le DOM complet de l'objet rempli pour le reconstruire a partir de la chaine. Même si le contenu de la chaine reprend les mêmes contenus plus autre chose
 - **`element.innerText='nouveau contenu text', element.textContent ='content'`**
 - » Aucune balise ne sera recrée par le dom, tout sera remplacé par des entités html
- Manipuler le DOM
 - Supprimer un élément de l'arborescence DOM
 - **`element.remove()`**
 - » L'élément supprimé existera toujours dans la variable élément avec tout sa constitution DOM. Juste il ne sera plus présent dans l'arborescence. Il pourra être remis dans le DOM au besoin



Manipulation DOM

- Manipuler le DOM
 - Ajouter un élément dans l'arborescence DOM
 - ***element.append(...nodes)***
 - » Ajout dans l'élément *element* à la fin de celui-ci le ou les nœuds fournis en paramètres
 - ***element.insertBefore(node,child)***
 - ***element.insertAfter(node,child)***
 - » Place un élément dans le parent ***element*** avant ou après le nœud *child* en paramètre
 - ***element.replaceChild(node,child)***
 - » Remplace un élément *child* par l'élément *node*



Manipulation DOM & attributs class & id

- Les classes
 - Manipulation de chaine avec **className** (getter/setter)
 - **`element.className="classA classB"`**
 - Manipulation par gestion de liste avec **classList**
 - **`element.classList.add('newClassToAdd')`**
 - Ajout d'une class
 - **`element.classList.remove('classToRemove')`**
 - Suppression d'une class
- Les id
 - Manipulation de valeur d'id (getter/setter)
 - **`element.id="newId"`**

Manipulation DOM & attributs

- Le champ d'élément ***attributes***
 - ***element.attributes.getNamedItem('attName')***
 - ***element.getAttribute('attName')***
 - ***element.getAttributeNode('id')***
 - ***element.attributes.getNamedItemNS('attName','NSUri')***
 - ***element.getAttributeNS('attName','NSUri')***
 - Récupère un attribut par son nom
 - ***element.attributes.setNamedItem('attName','value')***
 - ***element.setAttribute('attName','value')***
 - ***element.attributes.setNamedItem('attName','value','NSUri')***
 - ***element.setAttributeNS('attName','value','NSUri')***
 - Met un attribut et sa valeur dans l'élément
- Manipuler les attributs par un Objet ***DOM attribute***
 - ***element.setAttributeNode(attNode)***



Manipulation DOM & attributs styles

- Les éléments possèdent un champs styles de type ***CSSStyleDeclaration***. Toutes les propriétés y sont présentes et peuvent être manipulées
 - Le nom des propriétés a été convertis en camel case
 - ***text-align*** deviens ***element.styles.textAlign***
 - Pour manipuler cette valeur il suffit de l'affecter
 - `element.styles.textAlign='center'`



Navigation dans le DOM

- Depuis un nœud, il est possible de naviguer dans son arborescence
 - Accéder au parent(getter)
 - ***element.parentElement***
 - ***element.parentNode***
 - » Donne l'élément/nœud parent du nœud élément
 - Accéder aux enfants
 - ***element.childNodes***
 - » nodeList des éléments (avec .forEach ...)
 - ***element.children***
 - » HTMLCollection des nœuds (sans .forEach,...)
 - Accéder aux adjacents
 - ***element.nextSibling***
 - ***element.previousSibling***
 - » Donne l'élément/nœud adjacent du nœud élément

8.FORMULAIRE & VALIDATION



Les formulaires

Structure des formulaires

- La balise `<form>` englobe tous les champs du formulaire
- L'attribut *action*
 - url de la page serveur qui réceptionne le formulaire
- L'attribut *method*
 - GET : envoie le contenu du formulaire condensé dans l'adresse url précédé par ? et séparés par &
 - a.php?champ1=value&champ2=val
 - POST : envoie dans le contenu du paquet http

```
<form action="a.php"
method="GET|POST">
  <input name="" type=""/>
  <textarea name="">
  </textarea>
  <input type='reset'/'>
  <input type='submit'/'>
</form>
```

détail : https://www.w3schools.com/tags/tag_form.asp

attributs

+target=" **_self** | _blank | _top | _parent | iframeName "

_self : dans la même page

_blank : nouvel onglet

+accept-charset=""

+autocomplete=" **on** | off "

+enctype=""

application/x-www-form-urlencoded

multipart/form-data

text/plain

Structure de Formulaire

- Les formulaires sont contenus dans une balise **form**

`<form name="monForm"><input name="c1"/></form>`

- Un tableau de tous les **form** du document est présente dans le champ **document.forms**
 - Il index par position DOM
 - » `document.forms[0]`
 - Il index par name de la balise form
 - » `document.forms['monForm']`
- L'objet DOM pour la balise **form** est aussi un tableau indexant tous les champs input/select la contenant
 - Il index par position DOM
 - » `document.forms[0][0]`
 - Il index par name de la balise form
 - » `document.forms['monForm']['c1']`

Input & js

- Modifié la valeur d'une entrée

- input

```
document.getElementById('input').value='nouveau content';
```

- select

```
document.getElementById('combo').selectedIndex=position;
```

- checkbox / radio

```
document.getElementById('radio').checked=1;
```

Contrôle des valeurs avant submit

- Vérification des champs obligatoires*

```
if( '' == document.getElementById('input-nom').value)
{
    alert('Saisir votre nom !');
    document.getElementById('nom').focus();
}
```

Privilégié
checkValidate()

- vérification des formats des valeurs **

```
var regex = /[\\w]+@[\\w]*.[\\w]{2,7}/;
if ( regex.exec(document.getElementById('mail').value)=== null)
{
    alert("saisir le mail");
    document.getElementById('date').focus();
}
```

- Vérification des interactions

- exemple date de naissance et checkbox +18ans

* html5 propose l'attribut required="required" sur les inputs, textarea, select
**html5 propose l'attribut patern="Regex" sur les inputs, textarea, et des types d'inputs mail, tel, ...

exemple interaction :

```
if(new Date(document.getElementById('dateNaiss').value).getFullYear()-(new Date()).getFullYear())>=18) {
    document.getElementById('radioMajeur').checked=1;
}
```

Formulaire et validation

- Les formulaires peuvent être soumis et reset par js :
 - `elemForm.submit()`
 - `elemForm.reset()`
- Des événements liés à ces actions sont disponibles
 - `elemForm.addEventListener('submit', (evt) => {...})`
 - `elemForm.addEventListener('reset', (evt) => {...})`
- Lors de la soumission un **`checkValidity()`** est déclenché et empêche la soumission en cas de non respect des valeurs il remplace les validations manuelles comme précédemment
 - Cette validation se base sur plusieurs choses :
 - Type d'input
 - Pattern
 - Length, ...
 - Au besoin cette vérification peut être déclencher sur les inputs lors de la saisie

```
elemInput.addEventListener("input", (evt) => {  
  if (evt.target.checkValidity()) {  
    evt.target.classList.remove("error"); // Vert  
  } else {  
    evt.target.classList.add("error"); // Rouge  
  }  
});
```

Validation et Messages

- La validation peut être vérifiée par :
elemInput.validity
- Cet objet de type ***ValidityState***, possède autant de champs que de règles de validation possible :
 - badInput, customError, patternMismatch, rangeOverflow, rangeUnderflow, stepMismatch, tooLong, tooShort, typeMismatch, valid, valueMissing
 - Ce champ est mis à jour à chaque caractère saisi
- Le message d'erreur peut être récupéré
 - Ne nécessite pas de ***checkValidity*** pour avoir le message d'erreur
 - ***elemInput.validationMessage***



Input et validation

- Les inputs possèdent des méthodes de validation interne
 - Types : number, email, tel, color, date, dateTime, ...
 - Attributs :
 - All : minLength, length, maxLength, required
 - Number : min, max
 - Texte :Pattern
 - » Permet la validation par regex



9.DONNÉES COTÉ CLIENT



WEBAPI storage

- Le navigateur vous propose plusieurs possibilités de stockage de données dans le navigateur
 - SessionStorage
 - Localstorage
 - Cookies
 - Base indexedDB
 - Websql (déprécié et disparu des navigateurs)
- Chacun est dédié un usage spécifique



Usage des API de persistance

- Cookies
 - Stocke des clefs ou des token de session
 - Sécurité :
 - Accès commun
 - Accessible a tous
- LocalStorage
 - Stocke des contenues pour assuré la persistance des datas d'une application et ou la mise en cache de données pour une présence a la réouverture
 - Sécurité :
 - Accès limité
 - Accessible uniquement dans l'app
- SessionStorage
 - Comme le localStorage mais sans persistance, pour la mise en cache ou le stockage de données le temps de la durée de vie de l'app dans le navigateur
 - Sécurité :
 - Accès limité
 - Accessible uniquement dans l'app



localStorage

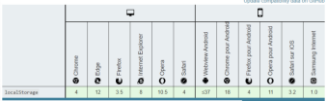
- Un stockage persistant dans le navigateur
- Accès limité a l'application web l'ayant créé
- Un couple propriété / valeurs
 - setItem

```
localStorage.setItem('monChat', 'Tom');
```
 - getItem

```
var cat = localStorage.getItem('monChat');
```
 - removeItem

```
localStorage.removeItem('monChat');
```
 - Clear

```
localStorage.clear();
```



Storage	Key	Value
localStorage	monChat	Tom

Documentation : https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API
<https://developer.mozilla.org/fr/docs/Web/API/Window/localStorage>

sessionStorage proche de localStorage

- sessionStorage sera détruit à la clôture de la session navigateur
- localStorage n'a pas de date de péremption

Le fonctionnement est similaire

Session storage

- LocalStorage et sessionStorage fonctionnent exactement pareil
- La seule différence réside dans la persistance de la data
 - localStorage
 - » Persistance complète même après fermeture du navigateur
 - » Destruction à assurer dans le code
 - sessionStorage
 - » Persistance limitée à la session
 - » Destruction automatique des données lors de la fermeture du dernier onglet de l'application web



Cookies

- *Propriété de document*

- document.cookie
- Accès possible par d'autres sites
- Utilisé par PHP pour relier la session client à des variables de session serveur
- path par def. = page courante
- une chaîne de caractères
- **Pensez à détruire vos cookies et ne stocker de des choses non critique**

- *Ecriture*

```
document.cookie = "username=John Doe";
```

```
document.cookie = "username=Nikola_TESLA; expires=Thu, 18 Dec 2050 12:00:00 UTC; path="/;
```

- *Lecture*

```
var cookie=document.cookie;
```

cookies

- Lecture directe

```
var cookie = new RegExp('; '+cookieName+'=(^\\s;]*)');
```

- fonctions de lecture

```
function readCookie(cookieName) {  
    var re = new RegExp('; '+cookieName+'=(^\\s;]*)');  
    var sMatch = (' '+document.cookie).match(re);  
    if (cookieName && sMatch) return unescape(sMatch[1]);  
    return '';  
}
```

- fonction d'écriture

```
function setCookie(cookieName,cookieValue,nDays) {  
    var today = new Date();  
    var expire = new Date();  
    if (nDays==null || nDays==0) nDays=1;  
    expire.setTime(today.getTime() + 3600000*24*nDays);  
    document.cookie =  
    cookieName+"="+escape(cookieValue)+";expires="+expire.toGMTString();  
}
```

10. JS MODERNE ET RESPONSIVE



Class

- Les classes
 - permettent la définition d'un objet
 - Des champs ou propriétés
 - Des méthodes du cycle de vie de l'objet
 - **constructor**,...
 - Des méthodes et des fonctions
 - Il peut avoir des champs qui lui appartiennent qu'à lui
 - Notion de **private**, **protected** (#)
 - Il peut avoir des champs/fonctions disponibles pour interagir avec lui
 - Notion de **public** / **private**

Génèrera une fonction pour l'es5 représentant l'objet transformé **sans class**

C'est l'histoire d'un mec...

- C'est l'histoire d'un mec, de 56 ans habitant à Mulhouse prénommé Jean-Pierre



- Il acheté une machine à café, le type

...

- Elle à un bouton marche / arrêt
- Un bac d'eau à remplir d'un litre
- Et du café en poudre à mettre dans le filtre à café

- Analysons la conception de l'objet



C'est l'histoire d'un mec...

- Sans intérêt pour nous

C'est l'histoire d'un mec, de 56 ans habitant à Mulhouse prénommé Jean-Pierre

- Identifions une class machine à café

- Des fonctions d'interaction **public**

- Elle a une fonction marche / arrêt
- Un champ définissant le volume d'eau du b
- Elle possède un champ de type café



- Des fonctions et champs qui lui sont propre

- Température de l'eau
- Activation de la résistance chauffante
- Activation de la pompe
- Eclairage du bouton lors du branchement

- Une cafetière c'est donc un objet!!!

aussi

Et Jean-P

Heu l'autre!!



Class machine à café

- Machine à café

- Des champs privés
- Des champs publics
- Un constructeur
- Des fonctions internes privées
- Des interactions extérieures, public
 - This, context de l'objet dans l'objet

```
class MachineCafe {
  #temperature: number;
  #temperatureMax=95;
  #isPumpStarted: boolean;
  #isHeaterStarted: boolean;

  isOnButton: boolean;
  cafe?: Cafe;
  niveauEau: number;

  constructor() {
    this.#temperature=0;... /*
    this.niveauEau=this.#senseWaterLevel();
  }

  #senseWaterLevel(){}
  #startPump(){}
  #startHeater(){}

  public makeCofee(){
    this.#startHeater();
    this.#startPump();
  }
};
```

C'est l'histoire d'un mec...

- Jean-Pierre n'est pas adroit il met du café en poudre partout tous les matins



- Il acheté une machine à café,
 - **oui encore**, système **café à dosettes** cette fois
 - elle **fait tout pareil**, mais plus pratique
- Elle à un bouton marche / arrêt
- Un bac d'eau à remplir d'un litre
- Et un objet **filtre** qui contient **café** est **redéfini** différemment
 - Il accepte désormais du **café en dosette**
- Alors une machine à dosette c'est une machine qui fait aussi du café.

Comme l'autre!!!



On parle ici de l'extension d'une class

qui reprend tout ce que fais un objet pour faire un nouvelle objet a partir du premier.

Il peut redéfinir des fonctions ou méthodes existante chez le parent et en apporter d'autres.

Ici notre cafetier possède les memes fonctions : faire le café par exemple

Mais le fonctionnement n'est pas profilement le meme.

Le café cette fois , toujours présent mais un nouveau conteneur doit etre insérer dans la machine , ... plus le café directement

Class machine à café

- Machine à café à dosettes
 - Ses champs privés
 - Ceux du **parent** étant eux aussi **private** il sont **inaccessibles**
 - » Notion d'héritage
 - » Notion de **protected**
 - Des champs / fonctions public, protected
 - Permettant l'héritage aux futures extensions
 - **Redéfinition** de la manière de faire le café
 - » Notions d'**override**
 - Des champs / fonction privés qui lui sont totalement personnel
 - Un constructeur
 - Appel du constructeur parent
 - » Notion de **super()**
 - Des fonctions interne protégé pour permettre aux futures générations de pleinement hérité de leurs ainés

```
class CoffeMachinPad extends CoffeMachin{
    public padHolder: PadHolder;

    constructor()
    {
        super();
    }

    public insertDose(pad:CoffePad){ ...
    }

    public makeCofe(){
        this.coffee= this.padHolder.coffee;
        this.startHeater(97);
        this.startPump(255);
    }
};
```

Nouvelle fonctionnalités ES6+

- Arrow function

- Remplace le mot clef *function*
- Permet de préserver le contexte de déclaration du mot clef *this*

» Ex : `const fn=(param1,)=>{...code...}`

- Template string

- Modèle de construction d'une chaîne de caractères délimité par ``` backtick (alt gr 7)
- Les contenus de variable ou d'appel de fonction js sont délimités par `${contenuDeValeur}`
- Les conteneurs de variable peuvent contenir des ternaires

» Ex : `const str=`partie static${variable} ${var1?var1:'content'}``



ES6+ déstructuration

- Déstructuration

- Permet de récupérer que certains champs d'un objet depuis les paramètres d'entrées d'une fonction

» Exemple :

```
const fn=({abc,fgh})=>{  
    console.log(abc,fgh);  
}  
fn({abc:123, xyz:234, fgh:'coucou')
```

ES6+ Spread operator

- Le spread operator permet
 - La création d'un nouvel objet peuplé par les valeurs d'une autre instance
 - Copie de surface
 - Itérable dans itérable, non itérable dans non itérable
 - Exemple :

```
const objet={abc:124, bcd:'une chaine', xyz:'coucou'}  
const nouvelleRef={...objet, xyz:888}
```

```
const array=[1,2,3,4]  
const nouvelleRefArray=[...array,5,6,7]
```

ES6+ type module

- En js l'ensemble des contenus chargés par la balise script est exécuté dans un même contexte d'exécution.
 - Tous les fichiers ont accès aux contenus des autres fichiers une fois chargés et exécutés
- Depuis ES6 la balise script (html) supporte l'attribut ***type="module"***
 - Délimite un contexte isolé d'exécution
 - Permet l'accès à tous les contenus non modules
 - Isole le code du module sans accès depuis l'extérieur
 - Notion d'import/export (cf. slide import export & scoping)



C'est l'histoire d'un mec ...

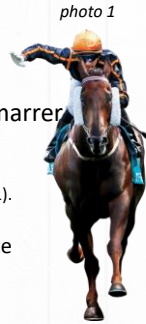
- Jean-Pierre lorsque il boit son café au bar joue souvent aux jeux d'argent.
- Plutôt chanceux, ce jour d'avril il se dit que c'est aujourd'hui

– Il gratte, il perd ..

– Il fait un loto

– Une **course de chevaux**, allait démarrer

- Il a donc fait un pari sur un cheval
 - Qui à un look de gagnant (cf. photo 1).
- Il attend devant la course qui débute
- Son cheval est placé
 - Mais il a coché sans le vouloir le
» *8 petit ange des prés* (cf. photo 2) → dernier
 - Enfin quand il arrivera, si il arrive



Ici une notion d'asynchronicité aurait pu être mise en place

Jp aurait pu attendre boire un petit café tranquillement ou tout simplement revenir demain

Peut être que tout sa vie reposais sur ce parie, il a donc tout perdu

Tout le reste de sa vie dépendais de ce paris il a donc attendu car il ne pouvait rien faire d'autre tant qu'il ne savais pas le déroulement de la suite des choses

C'est l'histoire d'un mec

- Jp sais qu'il a tout joué au tiercé du coup il rentre pu un sous en poche
- A 22h ce jour d'avril, il entend le tirage du loto
 - Souvenez vous, jp à joué avant le tiercé :
 - Il avait fait sa grille
$$3 - 8 - 21 - 35 - 49$$
 - Enregistrer son ticket contre un reçu de jeu
 - Il a donc prévu après le tirage de vérifier si il a gagner
 - » Il à déjà prévu que si il gagnait, il irait encaisser ses gains
 - et que c'est un gros lot, il déménagerait à Mulhouse
 - » Il a aussi prévu que si il gagnait rien, il jetterait son ticket
 - Vue qu'il étais d'humeur gagnante il a jouer et fait d'autre chose
 - C'est la notification de l'appli qui lui à rappeler que le tirage venait d'être fait
- Jp 42 ans en ce jour d'avril avais tous les bons numéros, dans l'ordre du loto.

Chanceux le type



technique

- Le jeu à gratter n'avait aucune notion d'async
- La course de chevaux n'a pas été utilisée sous forme d'async
- Notion de promise es6 et **await** fonction es7/ts
- Le loto quand à lui :
 - Jouer une fonction **async** qui **await** le tirage du soir
 - Une fois que l'action de jouer est finie j'exécute une tâche
 - En sachant à l'avance quelle prendra du temps et que j'aurais bien d'autres tâches à effectuer d'ici le résultat de la fonction appelée.
 - Je prévois directement, au moment de l'appel de la fonction, le **retour d'une promise**.
 - Promesse à laquelle je peux définir quoi faire quand l'exécution sera effectuée
 - avec `.then(retour=>{ ... });`



ES6+ Promise

```
• Fichier.js  
  
const promise=fetch('url')  
  
const treated=  
  promise.then(response=>response.json())  
treated.then(array=>console.log(array))  
  
const timeout=new Promise(resolved=>{  
  setTimeout(()=>resolved('charge',1000)  
})  
  
Promise.race([treated,timeout])  
  .then(res=>console.log(res))
```

- La promise est une promesse d'exécution lorsque la tâche async sera finalisée
 - Fetch basée sur les promises
 - Then renvoi aussi une promise
 - Chargé de la valeur de retour de la fonction en paramètre
 - Déclenché lorsque la tâche est :
 - resolved(fulfilled) (même si déjà résolue)
 - rejected
- Les promises peuvent être:
 - Mise en concurrence Promise.race
 - Attendu par ensemble: Promise.all

ES6 et appelle HTTP

- A l'instar de ES5 et *XMLHttpRequest* ES6+ redéfinit les appelle http grâce à *fetch* basé sur le system de *promise*
- Comme toute promise il est *awaitable*
- La promise de *fetch* ne retourne pas le contenu de réponse en charge mais un objet de type *Response* possédant un *stream* du contenu de réponse
 - Le *stream* doit être transformer dans le 1^{er} *then* puis préserver car un seul parcours est possible du *stream*
 - 3 manières de parcourir le *stream* :
 - *.blob()* pour les contenus binaires
 - *.text()* pour les contenus text génériques (html par ex.)
 - *.json()* pour les contenus json il seront parsé et retourné en instance js

```
const promise =  
  fetch("url", {  
    method: "PUT",  
    headers: {  
      "Content-Type":  
        "application/json",  
      Accept:  
        "application/json",  
    },  
    body:  
      JSON.stringify(uneInstance),  
  }).then((response) =>  
    response.json());  
  
promise.then((obj) =>  
  console.log(obj));
```

ES7+ Async / await / Promise

```
• Fichier.js  
  
async function doSomething2() {  
  console.log(await doSomethingAsync())  
}  
  
function doSomethingAsync() {  
  return new Promise((resolve) => {  
    setTimeout(() => resolve('i did'), 3000);  
  });  
}  
  
async function doSomething() {  
  console.log(await doSomethingAsync())  
  console.log(await doSomething2())  
}  
  
console.log('Before');  
doSomething();  
console.log('After');
```

- Pour être attendu une fonction :
 - Doit soit être déclarer **async**
 - Comporter un/des **await**
 - Doit retourner une promise
- Une fonction **async** s'utilise soit
 - » Notion d'attente explicite, préfixer par **await**
 - » Pas d'attente explicite, pas de préfixe à l'appel

Principe de responsive design

- En fonction des dimensions d'affichages:
 - Les composants doivent avoir des dimensions et / ou displays différents
 - Query
 - » Media : print, screen, speech*, all
 - » Conditionals:
 - width, max-width, min-width
 - height, max-height, min-height
 - orientation, résolution
 - and , or, not
 - » Exemple de media query:
only screen and (min-width: 768px)
screen and (orientation: portrait)



Principe de responsive design

- En fonction des dimensions d'affichages:
 - Les composants doivent avoir des dimensions et / ou displays différents
 - Css et media queries
 - Dans les balises link

```
<link rel="stylesheet" media="query"/>
```
 - Dans le css

```
@media query{  
    //...contenu css  
}
```
 - En js

```
var x = window.matchMedia("(max-width: 700px)");  
function myFunction(x) {  
    if (x.matches) { // If media query matches  
        document.body.style.backgroundColor = "yellow";  
    } else {  
        document.body.style.backgroundColor = "pink";  
    }  
}
```

11.OPTIMISATION



Optimisation DOM

- Privilégier ***.textContent*** a ***.innerHTML***
 - Plus rapide et évite les faille xss
- Privilégier les ***documentFragment*** pour des listes d'éléments a insérer dans le DOM
 - limite les rendus à chaque insertion d'élément
- Stocker les résultats de ***querySelector***
 - plutôt que de resélectionner le même élément dans une boucle



Optimisation de code

- Toujours mettre les **{...}** pour les blocs d'instruction (if, for, ...) même si il n'y a qu'une instruction
 - Accélère le parsing du code js
- Utiliser les fonctions de l'array **.map**, **.forEach**, ou les boucles **for..of**
 - Evite les recalcul de taille d'array
- Ne plus utiliser **var**, mais **const** ou **let**
 - Privilégier **const** par défaut



Optimisation chargement js dans le DOM

- Privilégier toujours le chargement de script dans html async
 - Evite de bloquer la construction du DOM pendant le chargement du script
 - Exécute qu'une fois le DOM chargé

`<script src="app.js" defer></script>`

- Privilégier le chargement dynamique des scripts grâce aux promise et import (ou grâce à *async await*)

`import('./module.js').then(...)`



Profilage & benchmark

- Il est possible de connaître le temps mis par du code à s'exécuter :

console.time('nom de mon chronomètre')

console.timeEnd('nom de mon chronomètre')

- Permet de démarrer un chronomètre puis de l'arrêter
- Plusieurs Chronos peuvent tourner en même temps

```
> console.time('c')  
> console.timeEnd('c')  
c: 0.001953125 ms
```

- Il est possible de logger avec la pile d'appelle

console.trace(value)

Cache et navigateur

- Un composant de stockage en cache pour les requêtes http existe dans le navigateur **cacheStorage**.

- Proche de la logique localStorage / sessionStorage mais pour la requête http

```
async function fetchFromCache(url) {  
  // 1. Ouvrir (ou créer) un espace de cache nommé "mon-cache-api"  
  const cache = await caches.open("mon-cache");  
  // 2. Vérifier si la requête est déjà dans le cache  
  const reponseCachee = await cache.match(url);  
  if (reponseCachee) {  
    console.log("Réponse depuis cacheStorage !");  
    return reponseCachee.json();  
  }  
  // 3. Sinon, faire la vraie requête réseau  
  console.log("Requête réseau en cours...");  
  const reponseReseau = await fetch(url);  
  // 4. Stocker une copie de la réponse dans le cache pour la  
  // prochaine fois  
  // Note : .clone() est obligatoire car le corps d'une réponse ne  
  // peut être lu qu'une seule fois  
  cache.put(url, reponseReseau.clone());  
  return reponseReseau.json();  
}
```

Import dynamique

- Import dynamique
 - Grace a async / await avec cache du module

```
const modules={}  
const lazyloaded=async (moduleName)=>{  
  if(module[moduleName])return moduleName  
  else {  
    const module = await import(moduleName); //  
    Dynamically load the module  
    modules[moduleName]=module.default(); // Access the default  
    export  
  }  
  return modules[moduleName]  
}  
lazyloaded('module').then(module=>module.fonction())
```

PROJET FINAL



Méthodologie

1. Définir les besoins
 - a. Découpage en fonctionnalités
 - b. Découpage en fenêtres logiques
2. Créer une maquette
 - a. Exporter en code
3. Choix techniques
 - a. Technologies à utiliser
 - b. Choix de serveur
 - c. Choix du domaine web (DNS)
 - d. Préparation d'un repos git
 - e. Organisation CI/CD
4. Dynamiser la maquette développement
 - a. Découper en bloc fonctionnels
 - b. Définir l'ordre de développements des fonctionnalités
 - c. Définir les besoins d'objets principaux
 - d. Structurer une documentation du projet pour le maintient
5. Tester les fonctionnalités
 - a. Tester
 - b. Compiler un distribuable & pousser les mise jours



Développement itératif

- Pour permettre une évolution continue du projet :
 - Privilégier un développement modulaire
 - Ajouter les fonctionnalités au fur et à mesure
 - Evolution progressive plutôt que tout d'un seul ensemble



Test et validation

- Pour assurer un projet de qualité
 - Prédéfinir un ensemble de test visuel pour s'assurer que l'application tourne sur toutes les machines
 - » Chrome permet la disposition sur plusieurs modes d'affichage
 - Tester chaque fonctionnalité
 - » Tester le bon fonctionnement
 - » Tester lorsque des erreurs sont fournis par l'utilisateur
 - » Vérifier le bon fonctionnement quelque soit l'action utilisateur



Test et validation

- Automatisation des tests et non régression
 - Test unitaires et test de navigateur
 - Vérifie à chaque demande que le code répond toujours correctement et comme attendu par les scénarios d'usage prédéfinis dans les sections de test
 - Permet d'automatiser les tests
 - » Web Test Runner / Modern Web
 - » Vitest (facilement intégrable dans un projet vite vanilla js)
 - » Cypress
 - » Playwright



Documentation de projets jsDoc

- Javascript possède un outil de documentation bien intégré dans les IDE le jsdoc et intégré dans certains outils pour faire des doc complètes (ex: compodoc)
- Chaque section jsdoc est isolé par

```
/**  
...contenu jsdoc...  
*/
```
- Les paramètres de jsdoc
 - Une liste complète de clefs pour chaque usage
 - @params
 - @returns
 - @extends
 - @type
 - @example
 - @todo
 - @throw
 - ...



CONCLUSION



Références

Navigateur :

Internet explorer <http://www.microsoft.com/france/windows/internet-explorer/>

Chrome <https://www.google.com/intl/fr/chrome/browser/?hl=fr>

Firefox <http://www.mozilla.org/fr/firefox/new/>

Safari <http://www.apple.com/fr/safari/>

Débugueur

Inclus dans chrome

Firefox <https://addons.mozilla.org/fr/firefox/addon/firebug/>

Internet explorer <http://www.microsoft.com/en-us/download/details.aspx?id=18359>

Editeur de texte / html

Notepad ++ <http://notepad-plus-plus.org/fr/>

Kompozer <http://www.kompozer.net/>

Visual studio code : <https://code.visualstudio.com/>

Plateforme node

Nodejs : <https://nodejs.org/fr>

Nvm : <https://github.com/nvm-sh/nvm> et <https://github.com/nvm-windows/nvm>

Références

Valider son code HTML : <http://validator.w3.org/>

Vérifier l'affichage d'une page : <http://screenqueri.es/>

Vérifier la compatibilité d'un navigateur avec HTML5 : <http://html5test.com/>

Vérifier le support d'une fonctionnalité dans un navigateur :

- <http://fmbip.com>
- <http://caniuse.com>
- <https://developer.mozilla.org/fr/>

Références

Livres de référence

- « **Bien développer pour le Web 2.0** » de Christophe Porteneuve chez Eyrolles.
- « **CSS avancées Vers HTML 5 et CSS 3** » de Raphaël Goetter chez Eyrolles.
- « **HTML5 pour les Web Designers** » de Jeremy Keith chez Eyrolles
- « JavaScript Avancé: Maîtriser DOM, async, web apps, optimisation, patterns et bonnes pratiques modernes » de Calen Kelmore
- « **DIVE INTO HTML5** » de Mark Pilgrim chez O'Reilly
(disponible en ligne sous licence CC-BY 3.0 : diveinto.html5doctor.com)

Sites de référence

- <http://www.w3.org>
- <http://www.w3schools.com>
- <http://devdocs.io>
- <http://www.html5doctor.org>
- <http://www.alsacreations.com>
- <http://developpez.net>
- <http://www.oreilynet.com>



Conclusion

Nous vous remercions d'avoir participé à cette formation.

Pendant cette formation, nous espérons :

- que vous avez appris de nombreux éléments théoriques et pratiques.
- que vous y avez trouvé plus que ce que vous étiez venu chercher.
- que vous vous sentez prêt(e) à relever les défis qui vous attendent.
- et que vous avez passé un bon moment avec votre professeur.



Conclusion finale

- Remerciements et encouragements

Pendant cette formation, nous espérons :

- que vous avez appris de nombreuses choses théoriques et pratiques
- que vous y avez trouvé plus que ce que vous étiez venu chercher
- et que vous avez passé un bon moment avec votre professeur

Merci d'avoir fait confiance à ORSYS, à bientôt pour d'autres formations !



152



A BIENTÔT



◀ FIN ▶

CAR + C'EST MIEUX

ANNEXES



DOM (Document Object Model)

Niveaux du DOM

- 0: Accès tabulaire aux données
- 1: Noyau d'interface (Core, html) pris en charge par les navigateurs récents
- 2: Modules Events, Style... (assez bien supporté dans l'ensemble)
- 3: Introduction de Load & Save, Validation et XPath et de fonctions XML natives
quelques options déjà implémentées (ex: XPath et XSLT sous MFF)

Concepts fondamentaux

- Connaître le DOM supporté par le navigateur permet de bien implémenter son code
- Modèle abrégé : #text... Noms en majuscules
- Nœuds, parents vs ancêtres, enfants vs descendants, frère précédent / suivant, valeur atomique (nœud sans enfant)

Remarque : Le DOM est une technologie XML. Un parser DOM est un analyseur syntaxique XML capable de lire et de transformer l'arbre de données.



DOM (Document Object Model)

4 propriétés fondamentales :

- `doctype`
- `documentElement`
- `Implementation`

- `nodeName`
- `nodeType` (`Node.ELEMENT_NODE` | `Node.ATTRIBUTE_NODE` | `Node.TEXT_NODE`)
- `nodeValue`
- `attributes`
- `parentNode`
- `childNodes`
- `firstChild`, `lastChild`
- `previousSibling`, `nextSibling` (n'oubliez pas les Nodes de whitespaces sous MFF !)

JAVASCRIPT HTML DYNAMIQUE

Node :

Méthodes de parcours :

- `parentNode` est `null` si on est sur le nœud `document` ou si le DOM n'a pas encore été construit (appel anticipé de méthode)
- `childNodes` renvoie un tableau vide `[]` s'il n'y a pas d'enfant (`childNodes.length == 0`) et `firstChild` / `lastChild` renvoient `null`
- `previousSibling` / `nextSibling` respectent l'ordre de création dans le document (le flux)

DOM (Document Object Model)

Les méthodes de lecture / modification :

- `hasAttributes()`
- `hasChildNodes()`
- `createElement(tagName)`
- `getElementById( idOfElement)`
- `appendChild( newChild)`
- `cloneNode(true/false)`
- `insertBefore( newNode, refNode)`
- `removeChild(refNode)`
- `replaceChild( newNode, refNode)...` (sous MSIE on a aussi `replaceNode`)

L'interface `Text` étend la capacité des nœuds pour les `TEXT_NODE`

JAVASCRIPT HTML DYNAMIQUE



Méthodes de modification:

- toutes les méthodes de création / substitution de nœuds suppriment le nœud existant si un nœud identique est inséré
- `appendChild( newChild)` ajoute le *newChild* à la fin (nouveau *lastChild*).
- `cloneNode( true/false)` le booléen indique si tout le nœud doit être copié (*true*) ou seulement sa structure (*false*) sans l'attacher au DOM.
- `insertBefore` insère un fils avant la référence. Quand cette référence est *null* équivalent à `appendChild`.

DOM (Document Object Model)

L'interface `Element` étend la capacité des nœuds pour les `ELEMENT_NODE`, propriétés et méthodes :

- `tagName`
- `hasAttribute( atrName)`
- `getAttribute( atrName)`
- `getElementsByTagName( tagName)`
- `removeAttribute( atrName)`

Certaines fonctions renvoient des collections de nœuds :

- `NodeList` : `item( index)`, `length`
- `NamedNodeMap`: `item + get/removeNamedItem( str)`, `setNamedItem( node)`



DOM (Document Object Model)

Module	Niv	Description
Core (noyau)	1	Interfaces fondamentales
HTML	1	Interface spécifique à (X)HTML dit DOMHtml
Views	2	Manipulation de la présentation (ex: CSS)
Events	2	Gestion des événements (écoute, propagation, traitement...)
Style	2	Manipulation des feuilles de styles
Traversal & Range	2	Parcours et filtrage du doc (traversal) et manipulation des fragments (range)
Load & Save	3	Stockage et récupération des documents
Validation	3	Validation du document par rapport à sa grammaire (DTD / Schema)
XPath	3	Extraction des nœuds à l'aide de XPath
Views & Formatting	3	Extension du module de Views
Abstract Schemas	3	Manipulation des grammaires (DTD / Schemas)



Un bistro service ?

- Un web service c'est comme
Un bistrot

- Il est de bon ton d'adopter le protocole adéquat
 - Bonjour



- Un serveur prend ma commande
 - Une bière pression
 - L'hydromel du patron est une bouteilleQui ne peut être servis aux clients



- Le serveur me sert ma commande
si cela est possible.
 - Je consomme ce qui m'a été servi dans
 - le verre adéquat
 - le volume parfait

- Dans certaines enseignes il est possible de savoir ce qui est possible de commander
- Il est même expliquer que
– Sur présentation d'un coupon Promo une réduction spéciale sera faite

- [illegible]

Un bistrot sur le web?

- Le bar symbolise:
 - Réserve
 - les données, sgbd, sensor, ...
 - Serveur du bar
 - comprendre une demande
 - Vérifier que le protocole & la demande sont bien formulés
 - Servir, traiter, assembler en fonction de la demande
 - Le client, nous humain,
 - machine consommant ce qui est servie en fonction du context
 - Quand à la carte
 - On parle ici de couche de **définition** de ce qui est **mis à disposition** ainsi que le **formalisme** pour le demander



Protocole http & REST

- Accès CRUD REST

- create
- Read
- update
- Delete

- Capacités

- Filtrage
 - champs
 - id
- Positionnement
- Relationnel *

Echanges
JSON

- Accès à une ressource par HTTP

- Méthodes HTTP

- POST (create)
- GET (read)
- PUT (update)
- DELETE (delete)

- Erreurs HTTP

- 2xx OK
- 4xx Erreur accès
- 5xx Erreur Serveur



163

